

COD-IV

Briefing do Projeto — Plataforma de estudo para exame de condução

Escola de Condução Auto-Marmindo

1. O que é

Site de estudo para exame de condução, desenvolvido para a Escola de Condução Auto-Marmindo. Serve para revisão rápida de conteúdo — código da estrada e mecânica — no estilo "simulador de exame", pensado para quem quer estudar em pouco tempo (scroll e responde, sem precisar de ler tudo).

Nome: **COD-IV** (Código + Ivandro).

2. Stack técnico

Camada	Tecnologia
Frontend	HTML / CSS / JS puro
Backend	Python (FastAPI), funções serverless
Base de dados	PostgreSQL centralizada (Vercel / Neon)
Hospedagem	Tudo no Vercel

Decisão importante: Java foi descartado do stack porque o Vercel não tem runtime nativo para Java (só Node.js, Python, Go e afins). Fica tudo em Python + HTML/CSS/JS.

Referência de estilo visual: <https://unc-two.vercel.app/> — layout limpo, toggle de tema claro/escuro, seleção por dropdowns, foco na função.

3. Fluxo do utilizador

- 1 Cadastro — primeiro nome + último nome + password (sem email). Username interno gerado automaticamente, não visível ao utilizador.
- 2 Login — nome completo + password.
- 3 Escolha de matéria — Código (sinalização) ou Mecânica.
- 4 Escolha do número de perguntas — 10, 15, 20 ou 50.
- 5 Temporizador — 1,5 minutos (90 segundos) por pergunta × número de perguntas escolhido.
- 6 Respostas — pergunta a pergunta, com opção de saltar e voltar depois. Pergunta saltada e não respondida no fim conta como errada.
- 7 Finalizar — botão para terminar o quiz.
- 8 Resultados — cada pergunta mostra se acertou/errou; se errou, mostra a opção certa.

Nº perguntas	Tempo total
10	15 min
15	22,5 min
20	30 min

50

75 min

4. Tipos de pergunta

Tipo	Aplica-se a	Formato	Interação
C — Reconhecimento	Código	Imagem do sinal + 4 nomes possíveis	Clica no nome certo (escolha única)
A — Afirmações c/ imagem	Só Mecânica	Imagem da peça + 4 afirmações (3 certas, 1 errada)	Clica nas afirmações certas (3 de 4)
B — Afirmações s/ imagem	Código e Mecânica	3 afirmações (2 certas, 1 errada)	Clica nas afirmações certas (2 de 3)

Regra de pontuação: pergunta só conta como certa se todas as opções certas forem marcadas e nenhuma errada for selecionada (tudo ou nada).

5. Schema da base de dados

utilizadores

id (PK), primeiro_nome, ultimo_nome, username (único, gerado), password_hash, criado_em

sinais

id (PK), codigo, nome, categoria, subcategoria, imagem_url, descricao

perguntas_mecanica

id (PK), pergunta, resposta_correta, categoria, imagem_url

quiz_perguntas

id (PK), tipo, materia, categoria, enunciado, imagem_url, referencia_id (FK)

quiz_opcoes

id (PK), pergunta_id (FK), texto, correta (boolean), ordem

quiz_sessions

id (PK), utilizador_id (FK), materia, num_perguntas, tempo_total_segundos, iniciado_em, finalizado_em, status

quiz_respostas

id (PK), quiz_session_id (FK), pergunta_id (FK), ordem, opcoes_escolhidas, correta, saltada

6. Conteúdo extraído

Fonte: manuais próprios do Danilo (Guia de Sinalização Rodoviária + Manual de Mecânica da Escola Auto-Marmindo).

sinais.json — 464 sinais extraídos, em 23 categorias: sinais de perigo, cedência de passagem, proibição, obrigação, seleção/afetação de vias, zona, informação, pré-sinalização, direção, confirmação, identificação de localidades, complementares, sinalização turístico-cultural, marcas rodoviárias, sinalização temporária, e símbolos (apoio ao utente, turísticos, geográficos/ecológicos, culturais, desportivos, industriais).

mecanica.json — 202 perguntas e respostas extraídas, em 14 categorias: partes do veículo, tipos de chassis, painel/iluminação, motor, sistema de distribuição, sistema de travagem, sistema de arrefecimento, sistema de lubrificação, sistema de transmissão, suspensão, direção, alimentação, alimentação diesel, sistema elétrico.

7. Estado atual do desenvolvimento

Concluído:

- Extração de conteúdo dos manuais (sinais + mecânica) — 464 sinais, 202 perguntas de mecânica

- Schema da base de dados definido e implementado (SQLAlchemy + SQL)
- Backend completo: cadastro, login, iniciar quiz, responder, finalizar, resultados
- Frontend completo: autenticação, seleção de matéria/nº perguntas, quiz com temporizador e navegação (incluindo saltar pergunta), ecrã de resultados
- Perguntas Tipo C (reconhecimento de sinais) geradas automaticamente — 464 perguntas
- Perguntas Tipo B (afirmações, código e mecânica) geradas automaticamente — 464 (código) + 202 (mecânica) = 666 perguntas
- Fluxo completo testado de ponta a ponta (cadastro → login → quiz misto código/mecânica → responder → finalizar → resultados) com sucesso

Pendente:

- Perguntas Tipo A (mecânica com imagem) — código já pronto, mas bloqueado: nenhuma das 202 entradas de mecanica.json tem imagem ainda
- Imagens dos sinais e das peças de mecânica — a fornecer pelo Danilo
- Definir COD_IV_ALLOWED_ORIGINS e COD_IV_SECRET_KEY nas env vars do Vercel antes de produção (ver secção 8)
- Deploy real no Vercel com Postgres/Neon ligado (usar connection string 'pooled', com -pooler no host)

8. Correções aplicadas na revisão de código

Revisão completa do projeto (backend, frontend, schema e dados) com as seguintes correções aplicadas e testadas:

- **Pool de ligações à BD em serverless** — o engine SQLAlchemy usava pool normal (QueuePool), que em funções serverless da Vercel esgota rapidamente o limite de ligações do Neon. Corrigido para NullPool + recomendação de usar a connection string "pooled" do Neon (com -pooler no host).
- **API deprecated do SQLAlchemy 2.0** — vários pontos usavam Query.get(), que está descontinuado. Substituído por Session.get().
- **Problema N+1 no ecrã de resultados** — o cálculo de resultados fazia uma query à BD por cada pergunta respondida (até 50 queries para um quiz de 50 perguntas). Corrigido para uma única query com IN.
- **CORS aberto a qualquer domínio** — passou a ser configurável via env var COD_IV_ALLOWED_ORIGINS, em vez de "*" fixo no código.
- **Chave secreta (JWT) por omissão** — o backend recusa arrancar em produção (VERCEL_ENV=production) se a env var COD_IV_SECRET_KEY não for definida, evitando assinar tokens com uma chave previsível.
- **Perguntas Tipo B implementadas** — geração automática de afirmações para código (categoria do sinal, certa/errada) e mecânica (par pergunta/resposta do manual, com a resposta errada vinda de outra pergunta da mesma categoria — distrator plausível sem inventar conteúdo).
- **Tipo A deixado pronto mas não corrido** — a função existe e corre automaticamente assim que mecanica.json tiver imagem_url preenchido; por agora está a 0/202 imagens, por isso não gera nada ainda.

Documento gerado como registo do estado do projeto COD-IV, para reutilização noutras conversas ou partilha com outros colaboradores.